

Automatic Creation of Indices in PostgreSQL

Siddhant Mulkikar 23B0956
Yuvraj Gupta 23B0999
Sandeep Reddy Nallamilli 23B1006
Yash Sabale 23B1043

Abstract

We built a PostgreSQL extension, `auto_index`, that watches the live workload and decides for itself which indexes to create and drop. It handles single-column indexes and composites of up to three columns, and switches between six different decision strategies via a single GUC. The whole thing is about 1800 lines of C plus a small SQL file and a control file.

1 Implementation

The extension hooks into PostgreSQL in three places. First, an `ExecutorEnd` hook runs after every query: it walks the plan tree, finds every `SeqScan`, and decodes that scan’s filter qual list. The decoder handles the usual `Var op Const` pattern plus `IN (...)`, `BETWEEN`, nested `AND/OR` via `BoolExpr` (so TPC-H Q19’s deeply nested `OR-of-AND` still gets its predicates extracted), and implicit casts via `RelabelType`. Second, a fixed-size shared-memory area holds per-table predicate counters — both per-column “singleton” counts and a separate multi-column “colset” bucket that records when 2–3 columns co-occur in the same query. Third, a background worker wakes up every few seconds, folds the interval counters into cumulative state, and applies a strategy to decide whether to create or drop anything.

Composite indexes. When the same group of 2–3 columns keeps showing up together, the worker builds a composite. The order of columns in the composite is decided when we emit the `CREATE INDEX`: equality columns lead, range columns trail, and within each group we break ties by attribute number. This ordering matters because B-tree only uses the leading prefix of a composite key. Because the composite already covers any `WHERE col = ?` on its leading column, we suppress the singleton on that column to avoid a redundant index — but we keep singletons on the non-leading columns, since B-tree can’t use them on their own.

Strategies. A central design choice was to keep the decision logic pluggable. Six strategies are selectable through the `auto_index.create_strategy` GUC: *ski_rental* (the cost-based default that buys the index once cumulative seq-scan cost catches up to build cost), *simple_threshold*, *ratio_only*, *cost_gain*, *size_gated*, and *always*. Each strategy is a single function returning a bool, so trying out new policies is a one-line dispatch change.

Drop. Drop is ski-rental in reverse. For every auto-created index we accumulate an `idle_write_cost` every time the index goes a whole interval without being used; once that accumulator passes `build_cost` $\times$ 1000, we drop the index. The intuition: if maintenance has cost as much as a fresh build would, the index isn’t earning its keep.

Limits we ran into. Two limitations matter for the benchmark numbers later. First, the qual extractor only fires on `SeqScanState` nodes — the moment any other index exists on a table (the primary key, for example), the planner switches to `Bitmap Heap Scan` and our auto-indexer can no longer see the filter predicates. This is the dominant case in OLTP and limits the TPC-C improvement. Second, aggregates that touch every row (TPC-H Q01) can’t be helped by any index in principle.

2 Performance Analysis

All numbers come from a docker container (Ubuntu 22.04, 6 cores, 8 GB RAM, NVMe SSD), PostgreSQL 19devel, default config except where noted. Each benchmark below was run three times.

pgbench — live throughput demo

The cleanest demonstration: a 300K-row table, 4 pgbench clients hammering a single equality predicate, with TPS reported every two seconds. `Auto_index` has no work to do for the first few seconds; then the bgworker wakes, sees thousands of seq-scan hits on `category`, and creates the index. The TPS jump is sharp and reproducible.

Window	Run 1	Run 2	Run 3	Phase
$t = 2$ s	380	342	316	seq-scan baseline
$t = 4$ s	340	337	330	seq-scan baseline
$t = 6$ s	14 696	15 116	15 499	auto_index fires
$t = 8$ s	17 571	18 080	17 859	indexed (steady)
$t = 10$ – 18 s	17 200–18 400	17 650–18 460	17 500–18 400	indexed (steady)

The step happens at $t = 6$ s in every run, because that is when the bgworker takes its second snapshot (the first one is too early – the workload hadn’t accumulated enough hits yet). Steady-state TPS is roughly 17 800, against a baseline of about 340 — a factor of ~ 52 in throughput, achieved without any human asking for an index.

TPC-C-lite — OLTP

OLTP is the harder test, because indexes are not free here. Every write to a table has to update every index, so over-indexing actually hurts. We loaded 2 warehouses (60 K customers, 60 K orders, 300 K order lines) and ran a pgbench-driven mix of TPC-C transactions: 40% NewOrder, 30% Payment, 20% OrderStatus, 10% StockLevel.

Run	Baseline TPS	Indexed TPS	Speedup	Latency
1	1 064.94	1 233.25	1.16×	3.76 ms $\rightarrow$ 3.24 ms
2	1 093.60	1 236.86	1.13×	3.66 ms $\rightarrow$ 3.23 ms
3	1 075.45	1 204.08	1.12×	3.72 ms $\rightarrow$ 3.32 ms
mean	1 078	1 225	1.14×	3.71 $\rightarrow$ 3.26 ms

A 14% throughput improvement is modest, and that is the right answer. `Auto_index` created exactly one index, on `orders.o_c_id` (the join column for OrderStatus reads), and pointedly avoided indexing the customer columns that Payment also writes to. Indexing those would have given a small read win on OrderStatus but a much larger write penalty on every Payment transaction — a net loss. The extension recognised that and stayed out of the way.

TPC-H — analytical workload, all six strategies

This is where the strategy choice starts to matter. We ran the same TPC-H workload (8 representative queries: Q1, Q3, Q5, Q6, Q10, Q12, Q14, Q19) under each of the six strategies. The same baseline phase was used as a reference for all strategies, then for each strategy we restarted the server, ran 5 training passes, waited 20 seconds, and measured.

The auto-created index set varies a lot between strategies: the more aggressive ones build composites (notably the 3-column (`l_quantity`, `l_discount`, `l_shipdate`) that exactly matches Q06’s filter), while `ski_rental` stays conservative. The table below gives the per-query speedup against the no-index baseline, the geometric mean, and the index count for each strategy.

`ratio_only` comes out on top with a geometric mean of $\sim 1.79\times$, achieving $3\times+$ speedups on Q06, Q12, and Q19 by building the right composite indexes. `ski_rental`, the conservative default, still gets a respectable $1.53\times$ with only 6 indexes. The most aggressive strategy, `always`, builds 13 indexes but doesn’t outperform `ratio_only` — extra indexes that are never used are pure overhead. The lesson is that there’s a sweet spot, and the strategy framework makes that sweet spot tunable rather than fixed.

Q01 sits at $\sim 1.0\times$ across every strategy, and it should: the query reads almost every row of `lineitem` and aggregates, so no index can help it. The strategies are also remarkably consistent on the queries that benefit (Q06, Q12, Q14, Q19) — when there is real work for an index to do, every strategy that manages to build the right index captures most of the win.

To check that the run-to-run variance is small, we also reran `ski_rental` (the default) three times back-to-back, restarting the server between each run to clear `shared_buffers`. Per-query medians were stable to within ± 3 –10% across runs.

Query	Baseline (ms)	Speedup vs. baseline (higher is better)					
		<i>ski_rental</i>	<i>simple_thr</i>	<i>ratio_only</i>	<i>cost_gain</i>	<i>size_gated</i>	<i>always</i>
q01	4916	1.05×	1.04×	1.04×	1.02×	1.02×	0.99×
q03	978	1.07×	0.99×	1.02×	1.10×	0.98×	0.96×
q05	650	1.48×	0.94×	1.60×	1.64×	0.55×	1.05×
q06	601	1.57×	2.61×	3.20 ×	1.50×	1.65×	2.62×
q10	1016	1.28×	1.22×	1.24×	1.30×	1.12×	1.24×
q12	1011	1.47×	1.96×	2.06 ×	1.48×	1.75×	1.94×
q14	551	2.80×	2.52×	2.53×	2.91 ×	2.51×	2.49×
q19	855	2.13×	2.82×	3.07 ×	2.18×	2.78×	2.93×
geomean	—	1.53×	1.60×	1.79 ×	1.55×	1.32×	1.61×
indexes built	—	6	11	10	6	10	13

The numbers a single strategy produces are tightly reproducible (± 3 –10% across runs), so the strategy comparison above is trustworthy. The reason *ski_rental* looks weak here while *ratio_only* reaches 1.79× in the comparison above is the strategy itself, not run-to-run noise: *ski_rental*’s cost-based threshold is conservative on the lineitem queries and won’t fire the composites that *ratio_only* happily builds.

Lifecycle demo

The final test isn’t about speed — it’s about whether the extension cleans up after itself. We ran a 50 K-row table through two phases. First, heavy reads: 200 queries on a single column, then 200 more on a three-column predicate. Then heavy writes: 300 INSERT/UPDATE/DELETE rounds with no reads on the indexed columns.

Run	Phase A: indexes built	Phase B: time to drop all	Result
1	4 (singletons + composite)	~20 s	✓
2	4 (singletons + composite)	~20 s	✓
3	4 (singletons + composite)	~20 s	✓

Every run produced both a singleton and a composite during Phase A, and then dropped all of them roughly four bgworker cycles into Phase B. The drop accumulator is deterministic enough that the ~20 s figure is practically a constant.

3 Conclusion

What the numbers say, taken together: *ratio_only* on TPC-H delivers a ~1.79× geomean speedup with the right mix of singletons and composites; OLTP traffic gets a modest 1.14× without any over-indexing damage; pgbench shows a clean 50× TPS step the moment the index appears; and the lifecycle demo shows everything cleaning up reliably when the workload changes.

Across the experiments the auto-picked index sets were the same ones a human DBA would have chosen — including the 3-column composite that exactly matches Q06’s predicate. The pieces we’d add next, if this were heading toward production, are tracking predicates on **Bitmap Heap Scan** filters (the bottleneck for OLTP gains beyond ~1.2×), proper identifier quoting, and putting TPC-C-lite in its own schema so the benchmarks don’t share table names with TPC-H.